Configuracion de Dependencias

Informe de Configuración Inicial: TicketAR - Mundial 2026

Fecha: 12 de Abril de 2026
Rol: Backend Engineering & DevOps

1. Visión General del Proyecto

TicketAR es la plataforma oficial de venta de entradas para el Mundial 2026, con un enfoque estricto en la seguridad, la eliminación de la reventa ilegal y la validación obligatoria con Pasaporte.

Reglas Críticas del Negocio:

- **Límite de Compra:** Máximo 1 entrada por sesión por usuario.
- **Reserva:** Bloqueo temporal del lugar (ej. 15 min) hasta la confirmación del pago.
- **Sectores:** Control de capacidad estricto (PLATEA, PALCO, POPULAR, PRENSA).

2. Reestructuración de Arquitectura (Monorepo)

Para cumplir con los lineamientos del documento fundacional (GEMINI.md), se corrigió la estructura de directorios del proyecto, adoptando un formato de monorepo ordenado:

- **apps/api-backend/**: Contiene la lógica del servidor (NestJS).
- **apps/web-frontend/**: Contiene la interfaz de usuario (Next.js + Tailwind).

Esta separación garantiza un aislamiento de dependencias y facilita la ejecución de comandos independientes para cada área del stack.

3. Entorno de Ejecución y Dependencias

Se reforzó el cumplimiento estricto del uso de **pnpm** como único gestor de paquetes permitido en el equipo:

- **Depuración:** Se eliminó el archivo npm-lock.yaml residual del backend para evitar conflictos y descargas erróneas por parte de otros desarrolladores.
- **Aislamiento:** La instalación de dependencias (pnpm install) debe ejecutarse exclusivamente dentro de las carpetas apps/api-backend o apps/web-frontend, nunca en la raíz.

4. Políticas de Seguridad y Repositorio (Git)

A. Configuración Global de .gitignore

Se implementó un archivo .gitignore global en la raíz del proyecto para asegurar el código y mantener el repositorio limpio de archivos no deseados. Las exclusiones principales incluyen:

- **Dependencias:** `node_modules/`
- **Archivos de Compilación:** `dist/, build/`
- **Credenciales:** `.env, .env.local, .env.production` (Previniendo la fuga de llaves privadas).

- **Logs y Caché:** Archivos generados dinámicamente que no aportan al código fuente.

B. Gestión de Archivos Locales

Se añadió la ruta documentacion/ al archivo .gitignore. Esta medida preventiva evita la exposición en el control de versiones de notas personales, diagramas temporales, borradores e informes de estado que son de uso estrictamente local para cada desarrollador y no conforman el código fuente principal de la aplicación.

5. Bitácora de Inicialización del Proyecto (Paso a Paso)

Para replicar este entorno desde cero o comprender el flujo de creación inicial, se siguieron los siguientes pasos arquitectónicos:

1. Preparación del Entorno Base

- Se verificó la instalación de **Node.js** (entorno de ejecución) y **pnpm** (gestor de paquetes).
- Se instaló globalmente el CLI de NestJS para facilitar la generación de código backend: `pnpm add -g @nestjs/cli`.

2. Creación del Repositorio y Arquitectura Monorepo

- Se inicializó un repositorio Git vacío (`git init`).
- Se definió la arquitectura del proyecto en el archivo fundacional GEMINI.md, estableciendo las reglas de negocio, el stack tecnológico (NestJS + Next.js) y la estructura de carpetas (apps/api-backend y apps/web-frontend).

3. Generación de los Subproyectos

- **Frontend:** Se inicializó el proyecto de Next.js con Tailwind CSS preconfigurado.
- **Backend:** Se generó la estructura base de NestJS utilizando su CLI (`nest new backend-nest`). Durante este proceso, se seleccionó **pnpm** como el gestor de paquetes por defecto.

4. Alineación con el Diseño Arquitectónico (Refactorización de Rutas)

- Se renombraron y movieron los directorios base (backend-nest y frontend-client) hacia la carpeta apps/ mediante comandos de Git (`git mv`), estableciendo las rutas definitivas: apps/api-backend y apps/web-frontend.
- Esta reestructuración garantizó el cumplimiento de la arquitectura monorepo definida en la visión del proyecto.

5. Limpieza y Estandarización del Entorno

- Se eliminaron archivos residuales de otros gestores de paquetes (ej. package-lock.json en el backend) para forzar el uso exclusivo de **pnpm**.
- Se implementó un archivo .gitignore global en la raíz para proteger credenciales (.env), ignorar carpetas pesadas (node_modules) y excluir la carpeta de borradores locales (documentacion/).

6. Instalación de Dependencias Core del Backend

Ubicados dentro de apps/api-backend, se instalaron las librerías críticas para la seguridad, validación y conexión con terceros:

- `@nestjs/config`: Para la gestión segura de variables de entorno.
- `class-validator` y `class-transformer`: Para la validación estricta de DTOs (Data Transfer Objects).

- `@supabase/supabase-js`, `stripe` y `mercadopago`: SDKs oficiales para base de datos, autenticación y pasarelas de pago.

Con estos cimientos, el proyecto quedó listo para iniciar el desarrollo de la lógica de negocio y los controladores.

Workflows y Versionados/Buenas Practicas en Equipo

Tipos de Versionado: (ELEGIR).

1. El Modelo de Forking (Lo que estás haciendo ahora)

Este es el estándar para **proyectos de código abierto** o equipos que quieren una separación total de responsabilidades.

- **Cómo funciona:** Cada desarrollador tiene su propia copia exacta del repositorio en su cuenta personal (el Fork).
- **Permisos:** Tú eres el dueño de tu Fork, así que puedes romperlo, borrar ramas o experimentar sin afectar a nadie.
- **Integración:** Cuando tu código está listo, pides permiso al dueño del repo original mediante un **Pull Request**.
- **Ideal para:** El proyecto del Mundial donde son 8 personas y quieren que el Project Lead revise todo antes de que entre a la rama principal.

2. El Modelo de Ramas Compartidas (Shared Repository)

Aquí no hay "copias" del repositorio. Todos trabajan sobre el mismo enlace de GitHub.

- **Cómo funciona:** Todos los integrantes del grupo tienen permisos de "Escritura" en el repositorio original. Simplemente clonan el repo de Juanma y crean ramas allí mismo ( ).
- **Riesgo:** Si alguien se equivoca y hace un a la rama por error, podría pisar el código de otro o romper la versión que funciona.
- **Ideal para:** Equipos muy pequeños (2 o 3 personas) que confían plenamente en el flujo del otro y necesitan velocidad máxima sin burocracia de forks.

Aunque el **Forking** es más seguro para aprender, para un proyecto universitario de 30 días, lo más eficiente suele ser el **Shared Repository con Protección de Ramas**:

1. **Todos en el mismo repo:** Todos clonan el de Juanma directamente.
2. **Ramas por tarea:** Nadie toca main. Tú creas feat/backend-tickets , otro crea feat/frontend-home.
3. **Protección:** Se configura GitHub para que **nadie** pueda subir cambios a main sin que otra persona apruebe su Pull Request.

1. Planificación y División de Trabajo (Avoid Conflicts)

Como bien dice tu nota, el conflicto surge cuando dos personas tocan lo mismo.

- **En el Backend:** Tú te encargarás de las APIs y la base de datos. Si otro compañero necesita tocar el backend, deben avisarse por **Discord o Whatsapp** (sus canales elegidos) para no trabajar en el mismo archivo al mismo tiempo.
- **Práctica sugerida:** Antes de programar, crea una **Issue en GitHub** para la tarea específica (ej: "Crear tabla de Tickets") y asígnatela a ti mismo. Así todos ven en qué estás trabajando.

2. Git LFS y Gestión de Archivos

Mencionaste el uso de **Git LFS** (Large File Storage).

- **Aplicación:** Si para tu backend necesitas archivos pesados (como un dump de base de datos de prueba o imágenes grandes para los sectores del estadio), no los subas directo a Git. Git LFS evita que el repositorio se vuelva lento y pesado.

3. Branch Protection y Pull Requests (Gatekeeping)

Esta es la práctica más importante para que el proyecto de **TicketAR** no falle.

- **Regla:** Nadie, ni siquiera el Project Lead, debería hacer push directo a .
- **El flujo:** Tú trabajas en tu rama ( ), haces el PR, y un compañero (ej. Juanma) debe revisarlo y aprobarlo. Si el código no está verificado, no se integra.

4. Comentarios a nivel de Proyecto vs. Subtareas

Tu nota dice: "Comment at the project level".

- **Por qué importa:** Si pones un comentario importante sobre la lógica de los pagos de **Stripe** en una subtarea perdida, nadie lo encontrará en dos semanas.
- **Dónde comentar:** Usa las **Discusiones de GitHub** o los comentarios en el **Pull Request** principal. Así queda un registro histórico centralizado.

5. Integración Continua (GitHub Actions)

Tu documento ya menciona el uso de **Github Actions**.

- **Automatización:** Configura una "Action" que ejecute y los tests de **Playwright** cada vez que alguien abra un PR. Si el código rompe el proyecto, GitHub no dejará que se haga el Merge.

Guía Maestra: Gestión de Monorepo, Dependencias y Flujo de Integración

En un proyecto de ingeniería, el repositorio de GitHub es la **"Receta"** (el código) y tu computadora es la **"Cocina"** (donde se ejecutan las dependencias).

1. ¿Qué se sube y qué no? (The Filter)

Es fundamental entender que **no todo lo que está en tu carpeta debe ir a GitHub**.

Elemento	¿Se sube (Push)?	Razón
Código Fuente (`.ts` , `.tsx` , `.css`)	**SÍ**	Es el trabajo real del equipo.
`package.json`	**SÍ**	Es la "lista de compras". Dice qué librerías necesita el proyecto para funcionar.
`package-lock.json`	**SÍ**	Asegura que todos instalen exactamente la misma versión de las librerías.
`node_modules/`	**NO**	Son archivos muy pesados. Cada uno los descarga localmente con `npm install` .
`.env`	**NO**	Contiene llaves secretas (Supabase, Stripe). Subirlo es un riesgo de seguridad.
`.env.example`	**SÍ**	Una plantilla vacía para que tus compañeros sepan qué variables configurar.

---

2. El Flujo de Integración (Cómo se une el trabajo)

Cuando tú (Backend) y tu compañero (Frontend) hacen push, GitHub no mezcla archivos binarios pesados, sino **líneas de texto** en los archivos de configuración.

El proceso paso a paso:

1. **Tu Parte:** Instalas Supabase en . Tu local cambia.
2. **El Push:** Subes ese a tu rama y luego haces el **Pull Request**.
3. **El Merge:** Juanma acepta tu PR. Ahora la rama del repo original tiene la "instrucción" de usar Supabase.

4. **La Sincronización:** El de Frontend hace . Su VS Code le avisará que el cambió.
5. **La Actualización:** Él ejecuta en su carpeta. Su computadora baja la librería que tú elegiste. **¡Ya están integrados!**

---

3. Buenas Prácticas de Configuración Local

Para que el equipo no pierda tiempo arreglando errores de entorno, sigan estas reglas:

Uso de : Crea un archivo en la raíz del backend que se vea así:

Fragmento de código

```
SUPABASE_URL=tu_url_aqui
SUPABASE_ANON_KEY=tu_key_aqui
STRIPE_SECRET_KEY=tu_key_de_prueba
```

Así, cuando alguien nuevo se sume, solo tiene que copiar ese archivo, renombrarlo a y poner los datos reales.

- **Instalación por Carpeta:**
  Como es un monorepo, cada uno debe ejecutar los comandos dentro de su carpeta de rol:
  -
  -

---

4. Checklist para antes de cada "Push"

Antes de enviar tus cambios al repositorio original, haz este repaso rápido:

1. **¿Compila?**: Ejecuta en el backend. Si da error, no lo subas.
2. **¿Linting?**: Revisa que el código esté ordenado (puedes usar).
3. **¿Mensaje Semántico?**: ¿Tu commit empieza con , o ?
4. **¿Archivos basura?**: Asegúrate de que no se te esté escapando un archivo temporal o un por accidente (revisa el comando).

INFORME CODIGO BACKEND

ESTRUCTURA /EXPLICACIÓN ARCHIVOS EN BACKEND.

ESTRUCTURA Y EXPLICACIÓN DE ARCHIVOS EN EL BACKEND.

📁 La Raíz (apps/api-backend/)

Estos archivos son el "motor" y la configuración de tu proyecto backend:

- **package.json**: El corazón del proyecto. Aquí se listan todas las dependencias (como NestJS, Supabase, Stripe) y los scripts para correr el servidor (ej: npm run start:dev).
- **nest-cli.json**: Configuración exclusiva de la herramienta de consola de Nest. Le dice a NestJS cómo construir y compilar tu proyecto.
- **tsconfig.json`` / ``tsconfig.build.json**: Las reglas estrictas de TypeScript. Configuran cómo el código TypeScript se traduce a JavaScript puro para que Node.js pueda ejecutarlo.
- **eslint.config.mjs`` / ``.prettierrc**: Tus "policías del código". ESLint busca errores de sintaxis y malas prácticas, mientras que Prettier se asegura de que el código esté formateado bonito y uniforme para todo el equipo.
- **.gitignore**: (El que movimos hace un rato). Le dice a Git qué archivos del backend **no** debe subir a internet (como la carpeta node_modules o tus .env).

📁 Carpeta Principal (src/)

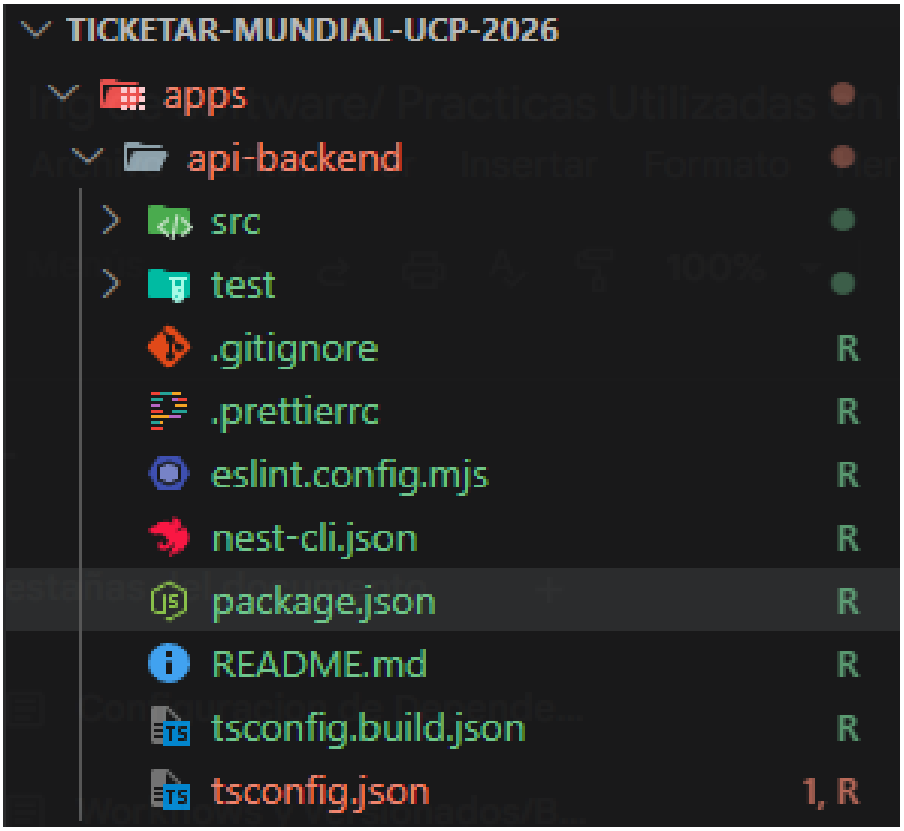
Aquí es donde vivirán tus módulos de negocio (Usuarios, Tickets, Pagos, etc.). Por ahora, NestJS te da los archivos base:

- **main.ts**: **El punto de entrada de tu aplicación.** Aquí es donde arranca el servidor, se define el puerto (usualmente 3000) y donde más adelante configuraremos reglas globales (como CORS, validaciones de DTOs, etc.). UNE TODOS LOS MÓDULOS SIGUIENTES.
- **app.module.ts**: **El módulo raíz.** Piensa en él como el tronco del árbol. A medida que crees módulos para "Tickets" o "Usuarios", los irás "importando" y conectando aquí para que NestJS sepa que existen.
- **app.controller.ts**: **El recepcionista.** Su trabajo es escuchar las peticiones HTTP (GET, POST, PUT, DELETE) que llegan desde tu frontend o Postman, y decidir a quién delegar el trabajo.
- **app.service.ts**: **El cerebro (Lógica de negocio).** El controlador le pasa la pelota al servicio. Aquí es donde harás las consultas a Supabase, validarás que el usuario no compre más de 1 ticket y procesarás los pagos.
- **app.controller.spec.ts**: **El archivo de pruebas unitarias.** Se usa con la librería Jest para probar *específicamente* que tu app.controller.ts funcione bien de forma aislada.

📁 Carpeta de Pruebas (test/)

En el documento GEMINI.md mencionaste que usarían Playwright para pruebas E2E (End-to-End), pero NestJS viene con Jest configurado por defecto para esto:

- **app.e2e-spec.ts**: A diferencia de las pruebas unitarias (que prueban partes aisladas), este archivo hace peticiones HTTP reales a tu servidor (simulando a un usuario) para ver si todo el flujo funciona de principio a fin.
- **jest-e2e.json**: La configuración que le dice a Jest cómo ejecutar esas pruebas E2E.



📁 Carpeta de Distribución (dist/)

La carpeta (abreviatura de *distribution*) contiene el **código compilado** de tu aplicación. Como estamos trabajando con NestJS y TypeScript ( ), Node.js no puede ejecutar ese código directamente. Cuando NestJS se "construye" ( ) o arranca, toma todo el código de la carpeta y lo traduce a JavaScript estándar ( ) para que el servidor pueda entenderlo.

Dentro de , encontrarás tres tipos de archivos por cada uno que tengas en :

- **Archivos .js (JavaScript):** Es tu código traducido. Es lo que realmente se ejecuta en el servidor.
- **Archivos .js.map (Source maps):** Son "mapas" que vinculan el código JavaScript con el TypeScript original. Esto es vital para que, si hay un error, la consola te indique exactamente dónde falló en tu código original ( ) y no en el compilado.
- **Archivos .d.ts (Declaraciones):** Archivos de tipos de TypeScript que sirven para que otras aplicaciones reconozcan qué tipos de datos usas.

En resumen: es donde **tú** trabajas; es lo que **la máquina** lee.

Migrar cambios desde un FORK a REPO ORIGINAL

Guía: Cómo migrar cambios desde un Fork a una rama específica en el Repositorio Original (Upstream)

Contexto: Estás trabajando en un Fork de un repositorio, en tu propia rama local (ej. feat/backend-logic), y necesitas subir ese código directamente al repositorio original (el del líder) en una rama con un nombre específico (ej. pzocikerwinBackend).

Pasos realizados:

1. **Verificar los repositorios remotos actuales:** Para asegurarte de qué repositorios tienes conectados actualmente (normalmente solo tendrás origin, que es tu fork).

```
git remote -v
```

2.
3. **Agregar el repositorio original como upstream:** Necesitas conectar tu entorno local con el repositorio original de tu líder. Lo llamaremos upstream.

```
git remote add upstream https://github.com/JuanmaInv/TicketAR---Mundial---UCP---2026-.git
```

4.
5. **Verificar que se haya agregado correctamente:** Vuelve a comprobar los remotos. Ahora deberías ver tanto origin (tu fork) como upstream (el original).

```
git remote -v
```

6.
7. **Empujar (push) tu rama local a la rama deseada en el upstream:** El comando mágico. Esto toma tu rama local (feat/backend-logic) y la sube al repositorio upstream, creando (o actualizando) allí una rama llamada pzocikerwinBackend.

```
git push upstream feat/backend-logic:pzocikerwinBackend
```

8.
9. *Nota: La sintaxis es git push <remoto> <rama-local>:<rama-remota>*

¿Cómo continuar trabajando a partir de ahora?

- **Para guardar más cambios en el repo del líder:** Haces tus commits normalmente en tu entorno local y, cuando quieras subirlos, vuelves a ejecutar el comando del paso 4: git push upstream feat/backend-logic:pzocikerwinBackend
- **Para actualizarte con el código de los demás:** Si el líder o el equipo hacen cambios en la rama main del repo original, puedes traerlos a tu entorno local con: git fetch upstream, git pull upstream main (o haciendo rebase/merge según prefiera el equipo).

🖥️ CONSTRUCCION DEL BACKEND

Estrategias de Organizacion.

🗺️ Guía General: Construir un Backend con NestJS desde Cero

🚀 Fase 1: Preparación del Terreno (Setup)

Antes de escribir lógica, configurás tu entorno de trabajo.

1. **Instalación del CLI:** Instalás el comando global de Nest para poder generar código automáticamente.

bash
```
pnpm add -g @nestjs/cli
```

2. **Creación del Proyecto:** Generás la estructura base. Nest crea el "esqueleto" de archivos necesarios.
3. **Limpieza inicial:** Eliminás los archivos de ejemplo generados automáticamente para entender cómo se conectan las piezas desde cero. Solo conservás y — el punto de entrada y la raíz del árbol de módulos.

📦 Fase 2: Modularización (La Estructura)

Aquí definís cuáles son las **unidades funcionales** de tu aplicación (cada recurso principal tendrá su propio módulo).

1. **Generar el Módulo:** Creás la "caja" que agrupa todo lo relacionado a esa función.

bash
```
pnpm nest g module <nombre-del-recurso>
```

2. **Generar el Controlador:** Creás el componente que escucha y responde peticiones HTTP (GET, POST, PATCH, DELETE).
3. **Generar el Servicio:** Creás el componente que contiene la lógica de negocio (guardar, transformar, eliminar datos).

🧬 Fase 3: Modelado del Dominio (Entidades)

Antes de procesar datos, definís **qué forma tienen** esos datos dentro de tu sistema.

1. **Crear la Entidad / Interfaz:** Escribís la clase o interfaz que representa tu recurso principal. Definís sus propiedades y sus tipos.
2. **Definir Estados (Enums):** Si alguna propiedad tiene un conjunto fijo de valores posibles, los encapsulás en un enum para restringir y documentar las opciones válidas.
3. **Tipado estricto:** Aprovechás TypeScript para que el compilador detecte errores de tipo antes de que lleguen a producción.

📝 Fase 4: Contratos de Entrada (DTOs)

Definís exactamente **qué datos esperás recibir** en cada operación — es la seguridad de tu frontera.

1. **Definir los DTOs:** Creás objetos que describen la forma exacta que deben tener los datos que envía el cliente (uno para crear, otro para actualizar, etc.).
2. **Agregar validaciones:** Usás para rechazar automáticamente peticiones malformadas (campos vacíos, tipos incorrectos, etc.) con un error .

🧠 Fase 5: Implementación de la Lógica (CRUD)

Implementás las operaciones fundamentales sobre tu recurso: **Create, Read, Update, Delete**.

1. **En el Servicio:** Escribís las funciones que manipulan los datos.
   - **Create:** Generás un identificador único (por ejemplo con) y persistís el nuevo registro.
   - **Read (All / One):** Devolvés la colección completa o buscás un registro por su ID.
   - **Update:** Localizás el registro y sobreescribís los campos indicados.
   - **Delete:** Eliminás el registro que coincide con el ID recibido.
2. **En el Controlador:** Conectás cada ruta HTTP con la función correspondiente del servicio, pasando los datos del request.

🧪 Fase 6: Pruebas y Verificación

Verificás que tu backend funciona **de forma independiente**, sin necesidad de que el frontend esté listo.

1. **Clientes de API:** Usás herramientas como Insomnia, Postman o Thunder Client para simular peticiones HTTP y observar las respuestas.
2. **Revisión de logs:** Monitoreás la consola para detectar errores en tiempo de ejecución y trazás el flujo de cada petición.

🏗️ Fase 7: Compilación y Despliegue (Build)

El paso final para llevar tu app al mundo.

1. **Build:** Compilás el proyecto — TypeScript se transpila a JavaScript optimizado en la carpeta .

bash
```
pnpm run build
```

2. **Producción:** El contenido de es lo que se sube a tu servidor o plataforma de hosting (Render, Railway, Supabase Functions, etc.).

📥 BACKEND(BOTTOMUP)

🗺️ Construir un Backend con NestJS BOTTOMUP

Fase 1: Preparación del Terreno (Setup)

Antes de escribir lógica, configurás tu entorno de trabajo.

1. **Instalación del CLI:** Instalás el comando global de Nest para poder generar código automáticamente.

bash
```
pnpm add -g @nestjs/cli
```

2. **Creación del Proyecto:** Generás la estructura base. Nest crea el "esqueleto" de archivos necesarios.
3. **Limpieza inicial:** Eliminás los archivos de ejemplo generados automáticamente para entender cómo se conectan las piezas desde cero. Solo conservás y — el punto de entrada y la raíz del árbol de módulos.

📦 Fase 2: Modularización (La Estructura)

Aquí definís cuáles son las **unidades funcionales** de tu aplicación (cada recurso principal tendrá su propio módulo).

1. **Generar el Módulo:** Creás la "caja" que agrupa todo lo relacionado a esa función.

bash
```
pnpm nest g module <nombre-del-recurso>
```

2. **Generar el Controlador:** Creás el componente que escucha y responde peticiones HTTP (GET, POST, PATCH, DELETE).
3. **Generar el Servicio:** Creás el componente que contiene la lógica de negocio (guardar, transformar, eliminar datos).

🧬 Fase 3: Modelado del Dominio (Entidades)

Antes de procesar datos, definís **qué forma tienen** esos datos dentro de tu sistema.

1. **Crear la Entidad / Interfaz:** Escribís la clase o interfaz que representa tu recurso principal. Definís sus propiedades y sus tipos.
2. **Definir Estados (Enums):** Si alguna propiedad tiene un conjunto fijo de valores posibles, los encapsulás en un enum para restringir y documentar las opciones válidas.
3. **Tipado estricto:** Aprovechás TypeScript para que el compilador detecte errores de tipo antes de que lleguen a producción.

📝 Fase 4: Contratos de Entrada (DTOs)

Definís exactamente **qué datos esperás recibir** en cada operación — es la seguridad de tu frontera.

1. **Definir los DTOs:** Creás objetos que describen la forma exacta que deben tener los datos que envía el cliente (uno para crear, otro para actualizar, etc.).
2. **Agregar validaciones:** Usás para rechazar automáticamente peticiones malformadas (campos vacíos, tipos incorrectos, etc.) con un error .

🧠 Fase 5: Implementación de la Lógica (CRUD)

Implementás las operaciones fundamentales sobre tu recurso: **Create, Read, Update, Delete**.

1. **En el Servicio:** Escribís las funciones que manipulan los datos.
   - **Create:** Generás un identificador único (por ejemplo con) y persistís el nuevo registro.
   - **Read (All / One):** Devolvés la colección completa o buscás un registro por su ID.
   - **Update:** Localizás el registro y sobreescribís los campos indicados.
   - **Delete:** Eliminás el registro que coincide con el ID recibido.
2. **En el Controlador:** Conectás cada ruta HTTP con la función correspondiente del servicio, pasando los datos del request.

🧪 Fase 6: Pruebas y Verificación

Verificás que tu backend funciona **de forma independiente**, sin necesidad de que el frontend esté listo.

1. **Clientes de API:** Usás herramientas como Insomnia, Postman o Thunder Client para simular peticiones HTTP y observar las respuestas.
2. **Revisión de logs:** Monitoreás la consola para detectar errores en tiempo de ejecución y trazás el flujo de cada petición.

🏗️ Fase 7: Compilación y Despliegue (Build)

El paso final para llevar tu app al mundo.

1. **Build:** Compilás el proyecto — TypeScript se transpila a JavaScript optimizado en la carpeta .

bash
```
pnpm run build
```

2. **Producción:** El contenido de es lo que se sube a tu servidor o plataforma de hosting (Render, Railway, Supabase Functions, etc.).

FLUJO DE DESARROLLO ARQUIT Y ORG BTMUP

INFORME PERTENECIENTE AL FLUJO DE DESARROLLO bottomUp CON LA ARQUITECTURA DE CAPAS.

ideas y desafios presentadas al desarrollador.

1) CAPA DE ORGANIZACIÓN: Modulos

Para crear nuestra app BACKEND, he implementado la modularización usando el principio de responsabilidad única S.O.L.I.D para separar en módulos cada uno de las funcionalidades del proyecto.

MatchesModule (Elección de partidos): Manejará los eventos, fechas y estadios.

SectorsModule (Visualización de asientos): Aquí controlaremos la capacidad estricta (PLATEA, PALCO, POPULAR, PRENSA) que menciona nuestro GEMINI.md.

ReservationsModule (Reserva): Este contendrá la regla de negocio más compleja: "El servidor bloquea el lugar por tiempo limitado (15 min) hasta confirmar pago".

PassportsModule o AuthModule (Pasaporte): Manejará la validación obligatoria y la regla estricta de "Máximo 1 entrada por sesión por usuario".

PaymentsModule (Pagos): La integración con Stripe/Mercadopago.

TicketsModule (Comprobante/Entrada): La generación del código QR y el envío por correo.

Esta arquitectura modular nos permitirá, por ejemplo, que si el día de mañana Mercadopago cambia su API, solo toquemos el PaymentsModule sin romper las reservas o los pasaportes.

CREACION DE MODULOS: Empezamos a crear cada módulo de las funcionalidades anteriores con la consola CLI. Estas se estructuran en carpetas individuales dentro del back y un **Módulo Raíz** el cual sería como un **Índice de todos los módulos**.

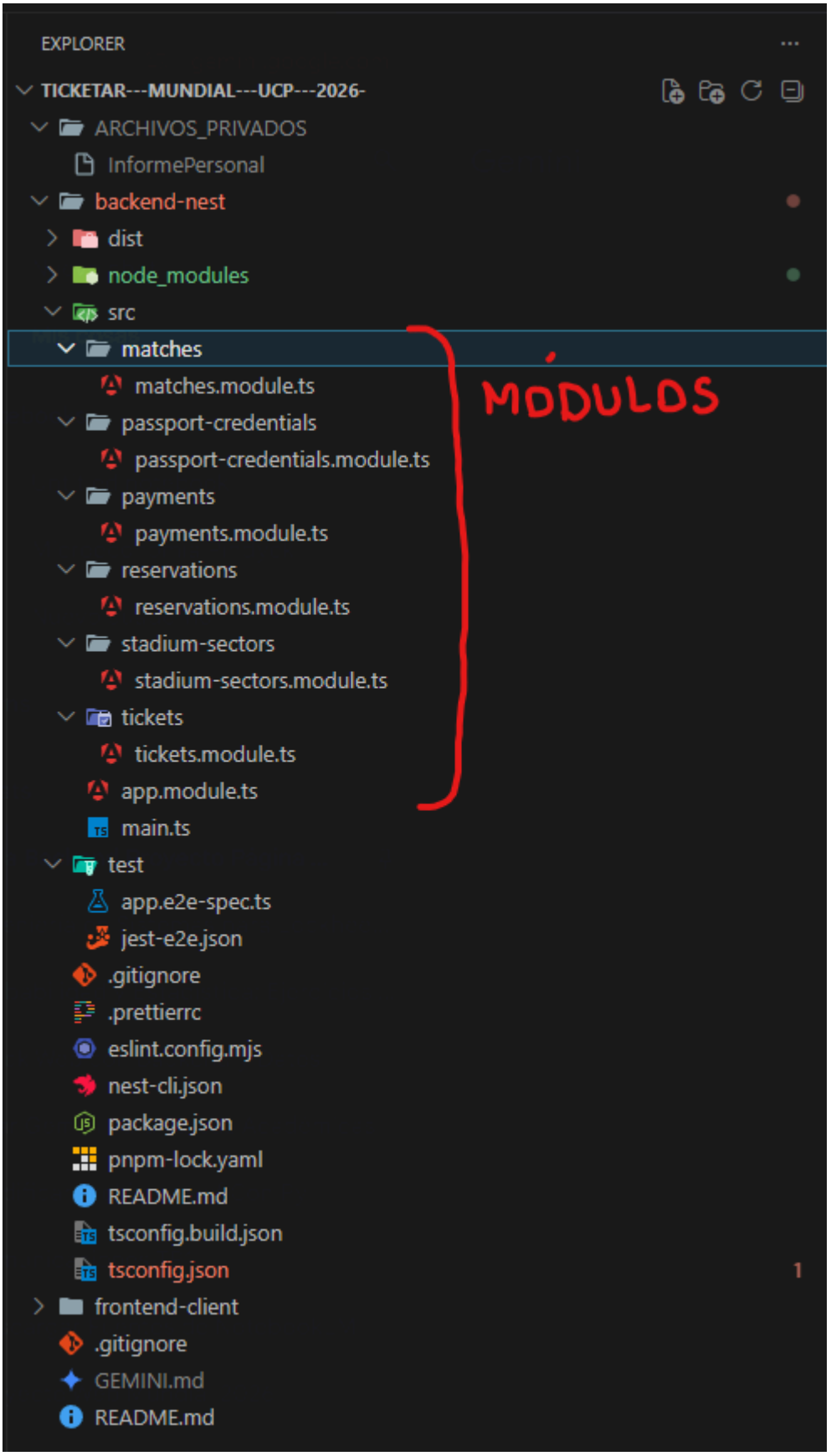

EXPLORER
TICKETAR---MUNDIAL---UCP---2026-
ARCHIVOS_PRIVADOS
InformePersonal
backend-nest
dist
node_modules
src
matches
matches.module.ts
passport-credentials
passport-credentials.module.ts
payments
payments.module.ts
reservations
reservations.module.ts
stadium-sectors
stadium-sectors.module.ts
tickets
tickets.module.ts
app.module.ts
main.ts
test
app.e2e-spec.ts
jest-e2e.json
.gitignore
.prettierrc
eslint.config.mjs
nest-cli.json
package.json
pnpm-lock.yaml
README.md
tsconfig.build.json
tsconfig.json
1
frontend-client
.gitignore
GEMINI.md
README.md
MÓDULOS

Imagen de Creación de módulos usando la consola.

```
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest> pnpm nest g module stadiumSectors
CREATE src/stadium-sectors/stadium-sectors.module.ts (95 bytes)
UPDATE src/app.module.ts (669 bytes)
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest> pnpm nest g module matches
CREATE src/matches/matches.module.ts (88 bytes)
UPDATE src/app.module.ts (742 bytes)
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest>
```

2) CAPA DE ENTRADA: Controlador
3) CAPA DE LÓGICA: Servicios

📤 BACKEND (TOPDOWN)

🧬 **Flujo de Diseño Orientado al Dominio (TOPDOWN)"**

Prioriza la Conectividad

- **¿Qué es?:** Prioriza la creación del Controlador y las Rutas (la cara externa).
- **¿Para qué sirve?:** Para que el equipo de Frontend pueda empezar a conectar la web rápido, aunque el "cerebro" (la lógica) todavía no esté terminado.
- **¿Por qué usarlo?:** Ideal cuando hay prisa por mostrar avances visuales o probar la comunicación entre la web y el servidor.

🏢 El Backend como un Edificio de Oficinas

Para que un edificio funcione, no puedes tener a todos los empleados en una sola habitación gritando órdenes. Necesitas Departamentos (Módulos), Recepcionistas (Controladores) y Especialistas (Servicios).

1. El Módulo (@Module): El Departamento
Un módulo es como el Departamento de Ventas o el Departamento de Seguridad.

Su función: Agrupa a la gente que trabaja en lo mismo.

En TicketAR: Tendrás un AuthModule (Seguridad), un TicketsModule (Ventas) y un EventsModule (Administración de estadios).

Informe: "Los módulos son unidades organizativas que encapsulan funcionalidades estrechamente relacionadas, permitiendo un código escalable y mantenible".

2. El Controlador (@Controller): El Recepcionista
Cada departamento tiene un recepcionista sentado en la puerta.

Su función: Escuchar quién viene de afuera (el Frontend) y qué quiere hacer. El recepcionista no hace el trabajo, solo recibe el pedido y dice: "Pase por aquí".

Los Métodos (GET, POST): Son las ventanillas.

GET: "Vengo a ver los precios".

POST: "Vengo a entregar mis datos para comprar".

Informe: "El controlador es el encargado de manejar las peticiones HTTP entrantes y dirigir las respuestas al cliente (Frontend)".

3. El Servicio (@Service): El Especialista

Detrás de la oficina, hay un especialista sentado en un escritorio con acceso a la caja fuerte (Base de Datos).

Su función: Él es quien realmente trabaja. Valida si hay stock, calcula el precio y guarda la información en Supabase. El recepcionista (Controlador) le pasa el papel con los datos, el especialista lo procesa y le devuelve el resultado.

Informe: "Los servicios contienen la lógica de negocio y son los encargados de interactuar con las fuentes de datos (repositorios/bases de datos)".Para que un edificio funcione, no puedes tener a todos los empleados en una sola habitación gritando órdenes. Necesitas **Departamentos** (Módulos), **Recepcionistas** (Controladores) y **Especialistas** (Servicios).

1. El Módulo ( ): El Departamento

Un módulo es como el **Departamento de Ventas** o el **Departamento de Seguridad**.

- **Su función:** Agrupa a la gente que trabaja en lo mismo.
- **En TicketAR:** Tendrás un (Seguridad), un (Ventas) y un (Administración de estadios).
- **Informe:** "Los módulos son unidades organizativas que encapsulan funcionalidades estrechamente relacionadas, permitiendo un código escalable y mantenible".

2. El Controlador ( ): El Recepcionista

Cada departamento tiene un recepcionista sentado en la puerta.

- **Su función:** Escuchar quién viene de afuera (el Frontend) y qué quiere hacer. El recepcionista **no hace el trabajo**, solo recibe el pedido y dice: "Pase por aquí".
- **Los Métodos (GET, POST):** Son las ventanillas.
    - **GET:** "Vengo a ver los precios".
    - **POST:** "Vengo a entregar mis datos para comprar".
- **Informe:** "El controlador es el encargado de manejar las peticiones HTTP entrantes y dirigir las respuestas al cliente (Frontend)".

3. El Servicio ( ): El Especialista

Detrás de la oficina, hay un especialista sentado en un escritorio con acceso a la caja fuerte (Base de Datos).

- **Su función:** Él es quien realmente trabaja. Valida si hay stock, calcula el precio y guarda la información en Supabase. El recepcionista (Controlador) le pasa el papel con los datos, el especialista lo procesa y le devuelve el resultado.
- **Informe:** "Los servicios contienen la lógica de negocio y son los encargados de interactuar con las fuentes de datos (repositorios/bases de datos)".

🛠️ El Flujo de Trabajo (Cómo se conectan)

Para tu informe, puedes describir el camino que recorre un dato cuando alguien compra una entrada para el Mundial:

1. **Petición:** El usuario hace clic en "Comprar". El Frontend envía una señal a .
2. **Módulo:** NestJS busca el para ver quién atiende ahí.
3. **Controlador:** El recepcionista del recibe los datos y verifica que el formato sea correcto (usando el **DTO** que vimos antes).
4. **Servicio:** El controlador le dice al : *"Oye, registra esta venta"*. El servicio verifica en Supabase si el asiento está libre y confirma la compra.
5. **Respuesta:** El controlador le devuelve un "¡Éxito!" al usuario.

📝 Resumen

Si tienes que escribirlo de forma académica para la **UCP**, puedes usar esta estructura:

> **Arquitectura Modular en NestJS**
>
> La arquitectura del backend se basa en el principio de **Responsabilidad Única**. Se divide en tres capas principales:
>
> - **Capa de Entrada (Controladores):** Punto de acceso para las peticiones externas.
> - **Capa de Lógica (Servicios):** Donde se procesan las reglas de negocio y validaciones.
> - **Capa de Organización (Módulos):** Contenedores lógicos que permiten la Inyección de Dependencias, facilitando que los controladores utilicen los servicios de manera eficiente.

4. La Entidad.

¿Qué es una Entidad en palabras simples?
Es una clase de TypeScript que sirve como el "molde" oficial de la aplicación. Gracias a esto, si definimos que una tarea tiene un título y una descripción, el sistema nos dará autocompletado y nos avisará si cometemos un error (como intentar poner un número donde va un texto).

Aquí es donde nos sentamos a pensar: *"¿Qué necesita una tarea?"*. Le asignamos un ID, un título, una descripción y un estado.

Uso de Enums (Estados fijos): En lugar de permitir que el estado sea cualquier palabra, usamos un . Esto es como una lista cerrada: la tarea solo puede estar "Pendiente", "En progreso" o "Hecha". Esto evita que alguien escriba mal un estado y rompa la lógica.
Instanciación: Al ser una clase, podemos crear "copias" de este molde (ej.).
Esto nos permite usar las propiedades de la entidad en cualquier parte del código con total seguridad.

Estas son el primer paso para conectar una **Base de Datos (como Supabase)**. Existen herramientas llamadas **ORMs** que leen estas clases y, automáticamente, crean las tablas en la base de datos por nosotros. Es decir, el código que escribimos en la entidad se convierte en la estructura real de nuestra base de datos sin tener que escribir lenguaje SQL complicado

INFORME DEFINITIVO

INFORME EN DESARROLLO

INFORME TÉCNICO DE DESARROLLO

TicketAR — Sistema de Venta de Entradas — BACKEND

Mundial 2026

Flujo de Desarrollo: Bottom-Up | Arquitectura: Capas

1. Estrategia de Desarrollo: Bottom-Up (SOLO BACKEND)

El equipo adoptó la estrategia Bottom-Up como enfoque principal de desarrollo. Tomamos esta decisión debido a que decidimos priorizar las capas del sistema — base de datos, lógica de negocio y servicios — antes de avanzar hacia la interfaz de usuario.

Justificación de la decisión

El sistema TicketAR involucra reglas de negocio complejas que deben estar sólidas antes de exponerse al cliente:

- El bloqueo temporal de asientos (15 minutos) durante el proceso de pago.
- La restricción estricta de máximo 1 entrada por sesión por usuario.
- La gestión de capacidad por sectores (PLATEA, PALCO, POPULAR, PRENSA).
- La integración con múltiples proveedores de pago (Stripe / MercadoPago).

Construir estas bases primero garantiza que el Frontend consuma una API ya validada y estable, evitando que errores de lógica se descubran recién en la integración visual.

2. Arquitectura de Capas

Se implementó una Arquitectura de Capas dentro del backend NestJS. Cada módulo del sistema respeta esta separación de responsabilidades, siguiendo el principio de responsabilidad única (S de SOLID).

2.1 Capa de Organización: Módulos

El backend se estructuró en módulos independientes, cada uno encargado de una funcionalidad específica del sistema. Estos módulos se crean con la CLI de NestJS y se registran en un Módulo Raíz (AppModule) que actúa como índice central.

Módulos implementados:

- **MatchesModule** *(Elección de partidos):* Maneja eventos, fechas y estadios disponibles.
- **SectorsModule** *(Visualización de asientos):* Controla la capacidad por sector: PLATEA, PALCO, POPULAR, PRENSA.

- **ReservationsModule** *(Reserva):* Contiene la regla de negocio más compleja: bloqueo de asiento por 15 minutos hasta confirmar el pago.
- **PassportsModule / AuthModule** *(Pasaporte / Autenticación):* Valida la identidad y aplica la restricción de máximo 1 entrada por sesión por usuario.
- **PaymentsModule** *(Pagos):* Integración con Stripe y MercadoPago.
- **TicketsModule** *(Comprobante / Entrada):* Generación del código QR y envío del ticket por correo electrónico.

Beneficio de esta modularización: *sí MercadoPago cambia su API, solo se modifica el PaymentsModule sin tocar reservas, autenticación ni tickets.*

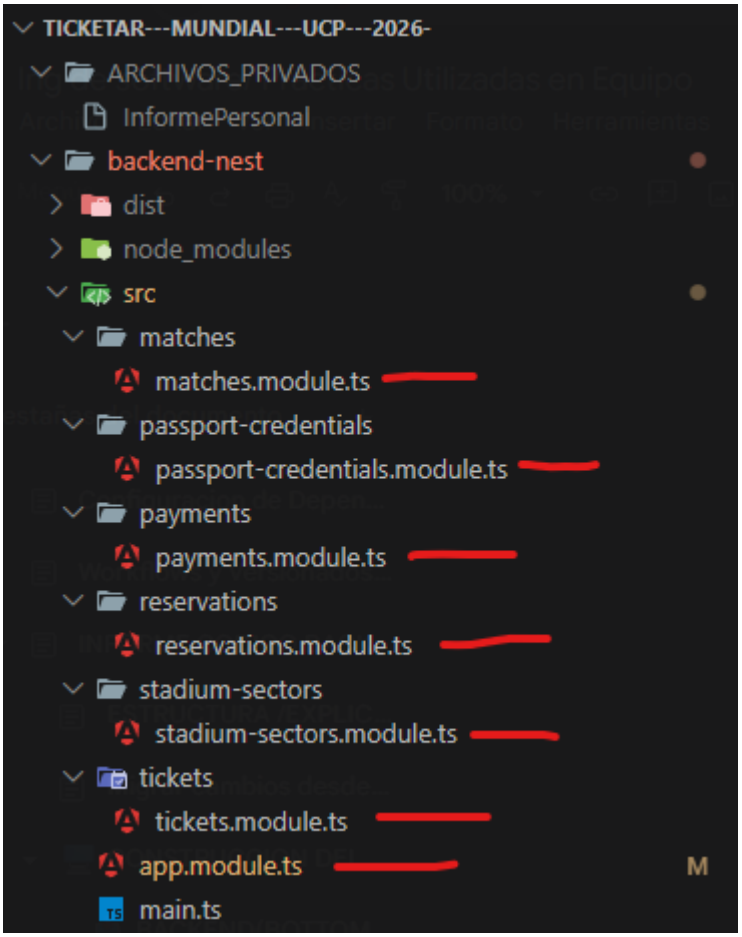
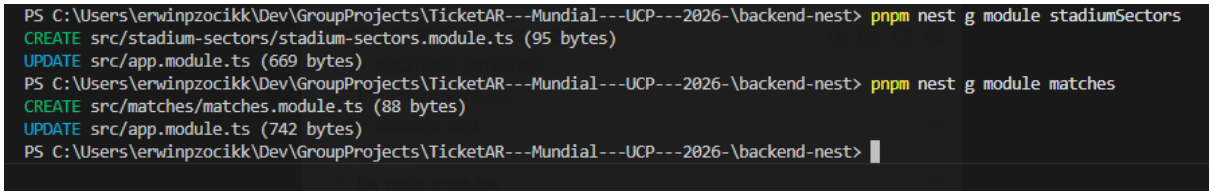


Imagen de Creación de módulos usando la consola.



```
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest> pnpm nest g module stadiumSectors
CREATE src/stadium-sectors/stadium-sectors.module.ts (95 bytes)
UPDATE src/app.module.ts (669 bytes)
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest> pnpm nest g module matches
CREATE src/matches/matches.module.ts (88 bytes)
UPDATE src/app.module.ts (742 bytes)
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest>
```

2.2 Capa de Entrada: Controladores

Los Controladores son el punto de entrada al sistema. Reciben las peticiones HTTP del cliente (GET, POST, PATCH, DELETE), validan los parámetros de entrada y delegan el procesamiento al Servicio correspondiente.

- No contienen lógica de negocio: solo coordinan la comunicación entre HTTP y Servicio.
- `Utilizan decoradores de NestJS: @Controller(), @Get(), @Post(), @Body(), @Param().`
- Cada módulo tiene su propio controlador (ej: ReservationsController, PaymentsController).

```
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest\src\tickets> npx nest generate controller tickets --flat

CREATE tickets.controller.ts (107 bytes)
CREATE tickets.controller.spec.ts (517 bytes)
UPDATE tickets.module.ts (248 bytes)
PS C:\Users\erwinpzocikk\Dev\GroupProjects\TicketAR---Mundial---UCP---2026-\backend-nest\src\tickets>
```

2.3 Capa de Lógica: Servicios

Los Servicios contienen toda la lógica de negocio del sistema. Son el núcleo de cada módulo y la razón principal por la que se eligió la estrategia Bottom-Up: deben estar completamente funcionales y testeados antes de exponer endpoints al frontend.

- Encapsulan las reglas críticas del dominio (bloqueo de 15 min, límite de 1 entrada por usuario).
- Se comunican con la Capa de Datos a través del patrón Repository (ver sección 3).
- Son inyectables: NestJS gestiona su ciclo de vida mediante Dependency Injection.

2.4 Capa de Datos: Repositorios y Entidades

Esta capa es responsable de la persistencia. Las Entidades definen el esquema de la base de datos y los Repositorios abstraen las operaciones de lectura/escritura. Los Servicios nunca acceden directamente a la base de datos.

- Entidades: Reservation, Match, Ticket, Payment, User (mapeadas con TypeORM).
- Repositorios: encapsulan las queries, implementando el patrón Repository (ver sección 3.1).

3. Patrones de Diseño Aplicados

Se implementaron cuatro patrones de diseño. En cada caso se justifica el problema concreto del sistema que resuelve, requisito de evaluación de la cátedra.

3.1 Patrón Repository

Módulo donde se aplica:

Todos los módulos, especialmente ReservationsModule y TicketsModule.

Problema que resuelve:

Los Servicios no deben acoplarse a la base de datos. Si en el futuro se migra de PostgreSQL a otro motor, la lógica de negocio no debería modificarse.

Cómo se implementa:

Se crea una clase ReservationsRepository que encapsula todas las operaciones de base de datos. El Servicio solo llama métodos como findActiveByUser() o blockSeat(), sin conocer detalles de SQL ni TypeORM.

Relación con SOLID: Implementa la D (Dependency Inversion): el Servicio depende de una abstracción (interfaz del Repository), no de una implementación concreta.

3.2 Patrón State

Módulo donde se aplica:

ReservationsModule.

Problema que resuelve:

La reserva pasa por múltiples estados: PENDING → CONFIRMED → EXPIRED / CANCELLED. La regla del bloqueo de 15 minutos es exactamente un problema de estados. Sin este patrón, el código tendría if/else anidados y sería propenso a errores de transición inválida (ej: confirmar una reserva ya expirada).

Cómo se implementa:

El estado de la reserva es verificado antes de cada operación. Cada transición válida está explícitamente definida. Si el estado actual no permite la acción solicitada, se lanza una excepción controlada.

Estados posibles:

- PENDING: asiento bloqueado, esperando pago (máx. 15 min).
- CONFIRMED: pago recibido, ticket generado.
- EXPIRED: tiempo agotado, asiento liberado automáticamente.
- CANCELLED: cancelado por el usuario antes del pago.

3.3 Patrón Strategy

Módulo donde se aplica:

PaymentsModule.

Problema que resuelve:

El sistema debe soportar dos proveedores de pago (Stripe y MercadoPago) con APIs distintas. Sin este patrón, el servicio tendría un if/else gigante por proveedor, y agregar un tercero requeriría modificar el código existente.

Cómo se implementa:

Se define una interfaz PaymentStrategy con el método processPayment(). Cada proveedor implementa esta interfaz de forma independiente: StripeStrategy y MercadoPagoStrategy. El PaymentsService elige la estrategia en runtime según la preferencia del usuario, sin conocer los detalles de implementación.

Vínculo con el diseño modular: *este patrón materializa exactamente el ejemplo mencionado en la Sección 2.1 — si MercadoPago cambia su API, solo se toca MercadoPagoStrategy.*

3.4 Patrón Observer

Módulos donde se aplica:

PaymentsModule → TicketsModule (comunicación entre módulos sin acoplamiento).

Problema que resuelve:

Cuando se confirma un pago, se deben disparar varias acciones: generar el ticket QR, enviar el correo al usuario y actualizar el estado de la reserva. Si PaymentsService llama

directamente a TicketsService y al servicio de correo, crea un acoplamiento fuerte y cualquier cambio en uno rompe al otro.

Cómo se implementa:

Con el EventEmitter2 de NestJS, PaymentsService emite un evento payment.confirmed al confirmar un pago. TicketsService escucha ese evento y reacciona generando el QR y enviando el correo. PaymentsModule no importa ni conoce a TicketsModule.

- Ventaja inmediata: agregar notificaciones push o analytics en el futuro no requiere tocar PaymentsService.
- Relación con la arquitectura: complementa la modularización — los módulos se comunican por eventos, no por dependencias directas.

4. Conclusiones y Estado Actual

La combinación de la estrategia Bottom-Up con la Arquitectura de Capas y los cuatro patrones de diseño genera un sistema coherente donde cada decisión técnica responde a un problema real:

- Repository desacopla datos y lógica.
- State modela el ciclo de vida crítico de las reservas.
- Strategy permite cambiar de proveedor de pago sin afectar el sistema.
- Observer desacopla la confirmación de pago de la generación de tickets.

El desarrollo avanza en el orden natural del flujo Bottom-Up: módulos → servicios → controladores → integración con frontend. La estructura modular garantiza que los cambios futuros (nuevos proveedores, nuevos sectores, nuevos tipos de entrada) se incorporen sin romper el sistema existente.

metodologias

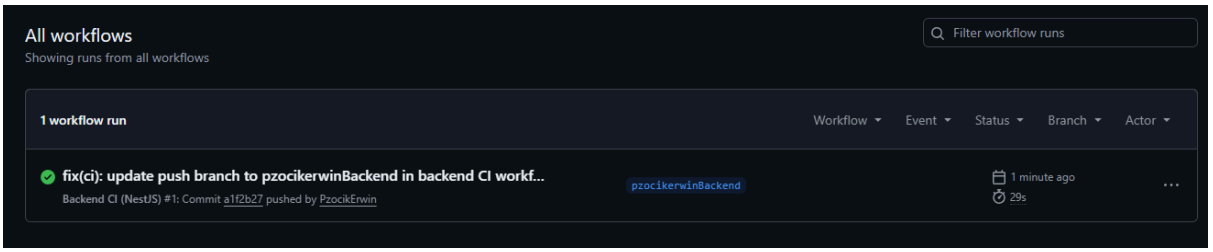
Pestaña 13

CONTINUOUS INTEGRATION (CI) GITHUB ACTIONS

Implementacion de ci mediante la carpeta .github\workflows donde se encuentra un archivo .yml el cual implementa la integracion en la rama main cada vez que se hace un push, esto se puede cambiar para evitar las restricciones de la rama especifica (implementar en todas las ramas o en una especifica que no sea el main).

```yaml
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

```

- **Al hacer push a pzocikerwinBackend**: Se ejecutará la CI para asegurarte de que tu código esté compilando correctamente (npm run build).
- **Al hacer push a main**: Ya **no** se ejecutará la CI automáticamente.
- **Al abrir un Pull Request hacia main (desde cualquier rama):** Se ejecutará la CI para validar el código antes de que se pueda fusionar con main (esta es una buena práctica de seguridad).

Aqui se puede evidenciar que el CI si aplico a mi rama de pzocikerwinBackend